

[1]Redis介绍

1.键值型与NoSql

Redis是一种键值型的NoSql数据库，这里有两个关键字：

- 键值型
- NoSql

其中**键值型**，是指Redis中存储的数据都是以key.value对的形式存储，而value的形式多种多样，可以是字符串.数值.甚至json：

而NoSql则是相对于传统关系型数据库而言，有很大差异的一种数据库。

对于存储的数据，没有类似Mysql那么严格的约束，比如唯一性，是否可以为null等等，所以我们把这种松散结构的数据库，称之为NoSQL数据库。

2. Redis数据类型

1 五种常用数据类型介绍

Redis存储的是key-value结构的数据，其中key是字符串类型，value有5种常用的数据类型：

- 字符串(string)：普通字符串，Redis中最简单的数据类型，string的内部结构实现上类似Java的ArrayList
- 哈希(hash)：也叫散列，类似于Java中的HashMap结构
- 列表(list)：按照插入顺序排序，可以有重复元素，类似于Java中的LinkedList,底层是双向链表
- 集合(set)：无序集合，没有重复元素，类似于Java中的HashSet
- 有序集合(sorted set/zset)：集合中每个元素关联一个分数(score)，根据分数升序排序，没有重复元素

1.zset的底层原理

Redis的有序集合（Zset）底层采用两种数据结构，分别是**压缩列表（ziplist）**和**跳跃表（skiplist）**。

- 当Zset的元素个数小于128个且每个元素的长度小于64字节时，采用ziplist编码。**在ziplist中，所有的key和value都按顺序存储，构成一个有序列表。这种实现方式主要是为了节省内存，因为压缩列表是一块连续的内存区域，通过紧凑的存储可以有效地利用空间。**虽然压缩列表可以有效减少内存占用，但在需要修改数据时，可能需要对整个列表进行重写，性能较低。
- 跳表是一种**多层次的链表结构，通过多级索引提升查找效率**。在不满足使用压缩列表的条件下，Redis会采用跳表作为Zset的底层数据结构。跳表能够**提供平均 $O(\log N)$ 的时间复杂度进行元素查找**，最坏情况下为 $O(N)$ 。**跳表中的每一层都是一个有序链表，并且层级越高，链表中的节点数就越少，从而允许在高层快速跳过一些元素，达到快速定位的目的。**

综上所述，Redis的Zset通过灵活地使用压缩列表和跳跃表作为底层数据结构，在不同的场景下平衡了内存使用效率和数据操作性能。这两种数据结构各有优劣，压缩列表适用于数据量小、内存受限的场景，而跳跃表适合于数据量大、需要高效操作的环境。

什么是跳表？

跳表（Skip List）是一种**基于有序链表的数据结构，通过多级索引的方式实现高效的查找、插入和删除操作。**

跳表以空间换时间的方式优化了传统单链表的效率。在单链表中，即使数据是有序的，查找一个元素也需要从头到尾遍历整个链表，时间复杂度为 $O(n)$ 。而在跳表中，通过建立多层索引来实现快速查找。顶层索引链表的节点数量远少于底层原始链表，并且层级越高，节点越少。

跳表中的每一层都是一个有序链表，并且每个节点都包含指向同层级下一个节点的指针以及指向下一层对应节点的down指针。例如，当查找一个元素时，首先在顶层索引进行查找，如果当前节点的值大于要查找的值，则继续在同一层级向右移动；如果小于要查找的值，则通过down指针下沉到下一层继续查找。每下降一层，搜索范围就缩小一半，最终在底层链表中找到目标元素或者确认元素不存在。

跳表的插入和删除操作同样高效，其时间复杂度也是 $O(\log n)$ 。向跳表中插入新元素时，首先要找到合适的插入位置，保持链表的有序性。然后通过随机函数决定新节点应该出现在哪些层级的索引中：随机结果高于某个固定概率 p ，就在该层级插入新节点。删除操作类似，先找到要删除的节点，然后在所有包含该节点的层级中移除它。

2.hash的底层原理

Redis的Hash数据结构底层原理主要基于两种数据结构：ziplist和hashtable。

具体来说，这两种数据结构的应用如下：

- **ziplist**：当满足特定条件时（键和值的字符串长度都小于64字节，且键值对数量少于512），Hash数据结构会采用ziplist作为其底层实现。在ziplist中，所有的key和value都按顺序存储，构成一个有序列表。这种实现方式主要是为了节省内存，因为压缩列表是一块连续的内存区域，通过紧凑的存储可以有效地利用空间。
- **hashtable**：当不满足ziplist的条件时，Hash数据结构会使用hashtable作为底层实现。在hashtable中，每个键值对都以字典的形式保存，其中字典的键为字符串对象，保存了原键值对的键；字典的值为另一个字符串对象，保存了原键值对的值。这样的结构允许快速的查找、插入和删除操作。

此外，在Hash数据结构中，如果ziplist编码所需的两个条件中的任意一个不再满足时，会发生编码转换，即原本保存在ziplist中的所有键值对会被转移到字典中，对象的编码也会从ziplist变为hashtable。这通常发生在键的长度过大、值的长度过大或者键值对的数量过多的情况下。

综上所述，Redis的Hash数据结构根据数据的规模和访问模式灵活地在ziplist和hashtable之间切换，以达到既节省内存又保证访问效率的目的。

2.3种redis特殊数据类型

Bitmap（位图）

Bitmap 存储的是连续的二进制数字（0 和 1），本来int数字占4字节32位，但通过 Bitmap, 只需要一个 bit 位来表示某个元素对应的值或者状态(比如:01表示1，001表示2)。，所以 Bitmap 本身会极大的节省存储空间。

```
# 将名为myBitmap的bitmap的第5位设置为1
SETBIT myBitmap 5 1 //SETBIT key offset value
```

你可以将 Bitmap 看作是一个存储二进制数字（0 和 1）的数组，数组中每个元素的下标叫做 offset（偏移量）。

HyperLogLog（基数统计）

HyperLogLog 是一种有名的基数计数概率算法，基于 LogLog Counting(LLC)优化改进得来，并不是 Redis 特有的，Redis 只是实现了这个算法并提供了一些开箱即用的 API。

Redis 提供的 HyperLogLog 占用空间非常非常小，只需要 12k 的空间就能存储接近 2^{64} 个不同元素。这是真的厉害，这就是数学的魅力么！并且，Redis 对 HyperLogLog 的存储结构做了优化，采用两种方式计数：

- **稀疏矩阵**：计数较少的时候，占用空间很小。
- **稠密矩阵**：计数达到某个阈值的时候，占用 12k 的空间

Geospatial (地理位置)

Geospatial index（地理空间索引，简称 GEO）主要用于存储地理位置信息，基于 Sorted Set 实现。

通过 GEO 我们可以轻松实现两个位置距离的计算、获取指定位置附近的元素等功能。

数值范围0-40亿的数如何排序(bitmap)

使用Bitmap进行排序是一种特殊的方法，适用于处理大量数据的排序问题，尤其是在内存有限的情况下。以下是使用Bitmap排序的步骤：

1. **初始化Bitmap**：根据数值范围创建一个足够大的Bitmap。由于数值范围是0-40亿，Bitmap的大小需要能够覆盖这个范围，即至少需要40亿位。
2. **标记数值**：遍历待排序的数值列表，将每个数值在Bitmap中对应的位置标记为1。例如，如果数值是5，则在Bitmap的第6位（从0开始计数）标记为1。
3. **按位输出**：按照Bitmap的顺序，输出所有标记为1的位置对应的数值，即可得到排序后的结果。

4.String 还是 Hash 存储对象数据更好呢？

- String 存储的是序列化后的对象数据，存放的是整个对象。Hash 是对对象的每个字段单独存储，可以获取部分字段的信息，也可以修改或者添加部分字段，节省网络流量。如果对象中某些字段需要经常变动或者经常需要单独查询对象中的个别字段信息，Hash 就非常适合。
- String 存储相对来说更加节省内存，缓存相同数量的对象数据，String 消耗的内存约是 Hash 的一半。并且，存储具有多层嵌套的对象时也方便很多。如果系统对性能和资源消耗非常敏感的话，String 就非常适合。

在绝大部分情况，我们建议使用 String 来存储对象数据即可！

5.购物车信息用 String 还是 Hash 存储更好呢？

由于购物车中的商品频繁修改和变动，购物车信息建议使用 Hash 存储：

- 用户 id 为 key
- 商品 id 为 field，商品数量为 value

那用户购物车信息的维护具体应该怎么操作呢？

- 用户添加商品就是往 Hash 里面增加新的 field 与 value；
- 查询购物车信息就是遍历对应的 Hash；
- 更改商品数量直接修改对应的 value 值（直接 set 或者做运算皆可）；
- 删除商品就是删除 Hash 中对应的 field；
- 清空购物车直接删除对应的 key 即可。

6.使用 Redis 实现一个排行榜怎么做？

Redis 中有一个叫做 `Sorted Set` 的数据类型经常被用在各种排行榜的场景，比如直播间送礼物的排行榜、朋友圈的微信步数排行榜、王者荣耀中的段位排行榜、话题热度排行榜等等。

相关的一些 Redis 命令：`ZRANGE`（从小到大排序）、`ZREVRANGE`（从大到小排序）、`ZREVRANK`（指定元素排名）。

7.Set 的应用场景是什么？

Redis 中 `Set` 是一种无序集合，集合中的元素没有先后顺序但都唯一，有点类似于 Java 中的 `HashSet`。

`Set` 的常见应用场景如下：

- 存放的数据不能重复的场景：网站 UV 统计（数据量巨大的场景还是 `HyperLogLog` 更适合一些）、文章点赞、动态点赞等等。
- 需要获取多个数据源交集、并集和差集的场景：共同好友(交集)、共同粉丝(交集)、共同关注(交集)、好友推荐（差集）、音乐推荐（差集）、订阅号推荐（差集+交集）等等。
- 需要随机获取数据源中的元素的场景：抽奖系统、随机点名等等。

8.使用 Set 实现抽奖系统怎么做？

如果想要使用 `Set` 实现一个简单的抽奖系统的话，直接使用下面这几个命令就可以了：

- `SADD key member1 member2 ...`：向指定集合添加一个或多个元素。
- `SPOP key count`：随机移除并获取指定集合中一个或多个元素，适合不允许重复中奖的场景。
- `SRANDMEMBER key count`：随机获取指定集合中指定数量的元素，适合允许重复中奖的场景。

9.使用 Bitmap 统计活跃用户怎么做？

Bitmap 存储的是连续的二进制数字（0 和 1），通过 Bitmap，只需要一个 bit 位来表示某个元素对应的值或者状态，key 就是对应元素本身。我们知道 8 个 bit 可以组成一个 byte，所以 Bitmap 本身会极大的节省存储空间。

你可以将 Bitmap 看作是一个存储二进制数字（0 和 1）的数组，数组中每个元素的下标叫做 offset（偏移量）。

如果想要使用 Bitmap 统计活跃用户的话，可以使用日期（精确到天）作为 key，然后用户 ID 为 offset，如果当日活跃过就设置为 1。

10.什么是HyperLogLog？使用 HyperLogLog 统计页面 UV 怎么做？

工作当中，我们经常会遇到与统计相关的功能需求，比如统计网站PV（PageView页面访问量），可以使用Redis的incr、incrby轻松实现。

但像UV（Unique Visitor，独立访客）、独立IP数、搜索记录数等需要去重和计数的问题如何解决？这种**求集合中不重复元素个数的问题称为基数问题**。

解决基数问题有很多种方案：

- （1）数据存储在MySQL表中，使用distinct count计算不重复个数
- （2）使用Redis提供的hash、set、bitmaps等数据结构来处理

以上的方案结果精确，但随着数据不断增加，导致占用空间越来越大，对于非常大的数据集是不切实际的。

能否能够降低一定的精度来平衡存储空间？Redis推出了HyperLogLog

Redis HyperLogLog 是用来做基数统计的算法，HyperLogLog 的优点是，在输入元素的数量或者体积非常非常大时，计算基数所需的空间总是固定的、并且是很小的。

在 Redis 里面，每个 HyperLogLog 键只需要花费 12 KB 内存，就可以计算接近 2^{64} 个不同元素的基数。这和计算基数时，元素越多耗费内存就越多的集合形成鲜明对比。

但是，因为 HyperLogLog 只会根据输入元素来计算基数，而不会储存输入元素本身，所以 **HyperLogLog 不能像集合那样，返回输入的各个元素。**

什么是基数？

比如数据集 {1, 3, 5, 7, 5, 7, 8}，那么这个数据集的基数集为 {1, 3, 5, 7, 8}，基数(不重复元素)为5。基数估计就是在误差可接受的范围内，快速计算基数。

使用 HyperLogLog 统计页面 UV 主要需要用到下面这两个命令：

- `PFADD key element1 element2 ...`：添加一个或多个元素到 HyperLogLog 中。
- `PFCOUNT key1 key2`：获取一个或者多个 HyperLogLog 的唯一计数。

1、将访问指定页面的每个用户 ID 添加到 HyperLogLog 中。

```
PFADD PAGE_1:UV USER1 USER2 ..... USERn
```

2、统计指定页面的 UV。

```
PFCOUNT PAGE_1:UV
```

11.redis怎么实现消息队列？

使用list类型保存数据信息，rpush生产消息，lpop消费消息，当lpop没有消息时，可以sleep一段时间，然后再检查有没有信息，如果不想sleep的话，可以使用blpop, 在没有信息的时候，会一直阻塞，直到信息的到来。

12.Redis 怎么实现延时队列

使用sortedset，拿时间戳作为score，消息内容作为key，调用zadd来生产消息，消费者用 `zrangebyscore` 指令获取N秒之前的数据轮询进行处理。

3. Redis常用命令

1 字符串操作命令

Redis 中字符串类型常用命令：

- | | |
|----------------------------------|------------------------------------|
| • SET key value | 设置指定key的值 |
| • GET key | 获取指定key的值 |
| • SETEX key seconds value | 设置指定key的值，并将 key 的过期时间设为 seconds 秒 |
| • SETNX key value | 只有在 key 不存在时设置 key 的值 |

setnx命令的原理

`SETNX` (SET if Not eXists) 是Redis提供的一个非常有用的命令，它允许用户在保证原子性的前提下为一个key设置值，但前提是这个key在Redis中尚不存在。以下是 `SETNX` 命令的工作原理：

1. **原子性**：Redis通过单线程模型执行命令，确保了 `SETNX` 操作的原子性。这意味着在并发环境下，同一时刻只有一个客户端能够成功地为一个key设定值。
2. **条件设置**：`SETNX` 命令只在给定的key不存在时设置其值。如果key已经存在，命令将不执行任何操作，并返回0。如果key不存在，命令将设置其值，并返回1。

setex命令的原理

Redis的SETEX命令用于将值设置为给定的key，并指定该键值对的过期时间。

Redis是一个高性能的键值存储系统，它支持多种数据类型和原子操作。其中，SETEX命令是用于设置一个带有过期时间的键值对。以下是SETEX命令的工作原理：

1. **键值存储**：在Redis中，每个key都与一个RedisObject结构相关联，该结构包含类型信息和指向底层简单动态字符串（SDS）的指针。当执行SETEX命令时，Redis会创建一个新的RedisObject来存储key，同时也会为value创建一个新的RedisObject。
2. **过期机制**：SETEX命令允许用户为键设置一个过期时间，这是通过在Redis的数据库结构中的expires字典中记录每个键的过期时间来实现的。这个过期时间是以毫秒精度的UNIX时间戳来表示的。
3. **原子性操作**：Redis通过其单线程模型确保了命令的原子性执行。

2 通用命令

Redis的通用命令是不分数据类型的，都可以使用的命令：

- KEYS pattern 查找所有符合给定模式(pattern)的 key
- EXISTS key 检查给定 key 是否存在
- TYPE key 返回 key 所储存的值的类型
- DEL key 该命令用于在 key 存在是删除 key

4.在Java中操作Redis

Spring Data Redis使用方式

Spring Data Redis中提供了一个高度封装的类：**RedisTemplate**，对相关api进行了归类封装,将同一类型操作封装为operation接口，具体分类如下：

- ValueOperations：string数据操作
- SetOperations：set类型数据操作
- ZSetOperations：zset类型数据操作
- HashOperations：hash类型的数据操作
- ListOperations：list类型的数据操作

StringRedisTemplate

尽管JSON的序列化方式可以满足我们的需求，但依然存在一些问题，如图：

为了在反序列化时知道对象的类型，JSON序列化器会将类的class类型写入json结果中，存入Redis，会带来额外的内存开销。

为了减少内存的消耗，我们可以采用手动序列化的方式，换句话说，就是不借助默认的序列化器，而是我们自己来控制序列化的动作，同时，我们只采用String的序列化器，这样，在存储value时，我们就不需要在内存中就不用多存储数据，从而节约我们的内存空间

这种用法比较普遍，因此SpringDataRedis就提供了RedisTemplate的子类：StringRedisTemplate，它的key和value的序列化方式默认就是String方式。

省去了我们自定义RedisTemplate的序列化方式的步骤，而是直接使用：

```
@Autowired
private StringRedisTemplate stringRedisTemplate;
```

```

@Test
void testString() {
    // 写入一条String数据
    stringRedisTemplate.opsForValue().set("verify:phone:13600527634", "124143");
    // 获取String数据
    Object name = stringRedisTemplate.opsForValue().get("name");
    System.out.println("name = " + name);
}

private static final ObjectMapper mapper = new ObjectMapper();

@Test
void testSaveUser() throws JsonProcessingException {
    // 创建对象
    User user = new User("虎哥", 21);
    // 手动序列化.将一个对象序列化为JSON字符串并存储到Redis中
    String json = mapper.writeValueAsString(user);
    // 写入数据
    stringRedisTemplate.opsForValue().set("user:200", json);

    // 获取数据
    String jsonUser = stringRedisTemplate.opsForValue().get("user:200");
    // 手动反序列化
    User user1 = mapper.readValue(jsonUser, User.class);
    System.out.println("user1 = " + user1);
}

```

最后小总结:

RedisTemplate的两种序列化实践方案:

- 方案一:
 - 自定义RedisTemplate
 - 修改RedisTemplate的序列化器为GenericJackson2JsonRedisSerializer
- 方案二:
 - 使用StringRedisTemplate
 - 写入Redis时, 手动把对象序列化为JSON
 - 读取Redis时, 手动把读取到的JSON反序列化为对象

[2].redis基础知识

0.redis怎么应对高并发?

在应对高并发的场景中, Redis作为一种高效的内存数据存储系统, 被广泛应用于缓存、消息队列等场景, 以提升系统的读取速度和降低数据库的负载。下面将逐一列出针对高并发问题时, 如何利用Redis来提高系统性能和稳定性:

1. 采用Redis集群模式

- **数据分片**: 在多个节点上分布数据分片, 即使某个节点不可用, 也不会影响其他节点的正常运行, 从而避免缓存雪崩现象。
- **跨机房部署**: 通过在不同地理位置的机房部署Redis集群, 可以进一步提升容灾能力, 保证在某个机房出现故障时依然能提供服务。

2. 使用持久化和预热缓存

- **数据持久化**：在重启Redis之前，通过执行SAVE指令将数据持久化到磁盘，确保数据不会因为重启而全部失效。
- **人工触发预热**：重启后手动或自动预热缓存，保证缓存在服务恢复后能够立即起到加速访问的作用。

3. 随机设置缓存过期时间

- **分散失效时间**：为缓存设置不同的过期时间，避免大量缓存在同一时刻集中失效，从而防止缓存雪崩的发生

0.怎么保证Redis的高并发高可用

首先可以搭建主从集群，再加上使用redis中的哨兵模式，哨兵模式可以实现主从集群的自动故障恢复，里面就包含了对主从服务的监控、自动故障恢复、通知；如果master故障，Sentinel会将一个slave提升为master。当故障实例恢复后也以新的master为主；同时Sentinel也充当Redis客户端的服务发现来源，当集群发生故障转移时，会将最新信息推送给Redis的客户端，所以一般项目都会采用哨兵的模式来保证redis的高并发高可用

1.redis为什么这么快？

嗯，这个有几个原因吧~~~

- 1、完全基于内存的，C语言编写,没有磁盘IO上的开销。数据存在内存中，读写速度快。
- 2、采用单线程，避免不必要的上下文切换以及锁等同步机制的开销
- 3、使用多路I/O复用模型，基于select/epoll等I/O多路复用技术实现高吞吐量网络I/O
 - IO多路复用是一种**操作系统中的一种技术，允许一个线程或进程同时处理多个输入输出（IO）操作**。Redis通过使用IO多路复用技术，以单线程的方式高效地处理了多个客户端的请求，避免了为每个连接创建新线程的开销，并保持高性能。

能解释一下I/O多路复用模型？

IO多路复用模型的思路就是：系统提供了select、poll、epoll函数可以同时监控多个fd(文件描述符)的操作，有了这个函数后，**应用线程通过调用select函数就可以同时监控多个fd，一旦某个描述符就绪（一般是读就绪或者写就绪），select函数就会返回可读/可写状态，这时询问线程再去通知想请求IO操作的线程，对应线程此时再发起IO请求去读/写数据。**

文件描述符是一个非负整数，用于标识被进程打开的文件，是操作系统为了高效管理这些文件所创建的索引。

为什么redis是单线程的？为什么后面又引入多线程？

12. 下面关于Redis选择单线程的原因，错误的说法是（）

- ☐ A 避免线程切换和竞争产生的消耗
- ☐ B 避免同步机制的开销
- ☐ C 实现简单，底层数据结构的设计无需考虑线程安全
- ☒ D 提升网络请求处理速度

正确答案：D

官方解析：

这道题考察的是Redis选择单线程的原因，Redis6.0版本引入了多线程的目的是解决Redis在网络 I/O 上的性能瓶颈，所以是D选项说法错误，A选项、B选项、C选项都是正确的说法。

2.既然Redis那么快，为什么不用它做主数据库，只用它做缓存？

虽然Redis非常快，但它也有一些局限性，不能完全替代主数据库。有以下原因：

事务处理：Redis只支持简单的事务处理，对于复杂的事务无能为力，比如跨多个键的事务处理。

数据持久化：Redis是内存数据库，数据存储在内存中，如果服务器崩溃或断电，数据可能丢失。虽然Redis提供了数据持久化机制，但有一些限制。

数据处理：Redis只支持一些简单的数据结构，比如字符串、列表、哈希表等。如果需要处理复杂的数据结构，比如关系型数据库中的表，那么Redis可能不是一个好的选择。

数据安全：Redis没有提供像主数据库那样的安全机制，比如用户认证、访问控制等等。

因此，虽然Redis非常快，但它还有一些限制，不能完全替代主数据库。所以，使用Redis作为缓存是一种很好的方式，可以提高应用程序的性能，并减少数据库的负载。

3.说一下 Redis 和 Memcached 的区别和共同点

现在公司一般都是用 Redis 来实现缓存，而且 Redis 自身也越来越强大了！不过，了解 Redis 和 Memcached 的区别和共同点，有助于我们在做相应的技术选型的时候，能够做到有理有据！

共同点：

1. 都是基于内存的数据库，一般都用来当做缓存使用。
2. 都有过期策略。
3. 两者的性能都非常高。

区别：

1. MemCached 数据结构单一，仅用来缓存数据，而 **Redis 支持多种数据类型**。
2. MemCached 不支持数据持久化，重启后数据会消失。**Redis 支持数据持久化**。
3. **Redis 提供主从同步机制和 cluster 集群部署能力**，能够提供高可用服务。Memcached 没有提供原生的集群模式，需要依靠客户端实现往集群中分片写入数据。
4. Redis 的速度比 Memcached 快很多。
5. Redis 使用**单线程的多路 IO 复用模型**，Memcached使用多线程的非阻塞 IO 模型。（Redis6.0引入了多线程IO，**用来处理网络数据的读写和协议解析**，但是命令的执行仍然是单线程）
6. value 值大小不同：Redis 最大可以达到 512M；memcache 只有 1mb。

4.Redis遇到哈希冲突怎么办？

当有两个或以上数量的键被分配到了哈希表数组的同一个索引上面时，我们称这些键发生了冲突（collision）。

Redis 的哈希表使用链地址法（separate chaining）来解决键冲突：每个哈希表节点都有一个 `next` 指针，多个哈希表节点可以用 `next` 指针构成一个单向链表，被分配到同一个索引上的多个节点可以用这个单向链表连接起来，这就解决了键冲突的问题。

原理跟 Java 的 HashMap 类似，都是数组+链表的结构。当发生 hash 碰撞时将会把元素追加到链表上。

5.redis线程模型

1.redis单线程模型

Redis基于Reactor模式开发了网络事件处理器，这个处理器被称为文件事件处理器。它的组成结构为4部分：多个套接字、IO多路复用程序、文件事件分派器、事件处理器。因为文件事件分派器队列的消费是单线程的，所以Redis才叫单线程模型。

- 文件事件处理器使用I/O多路复用（multiplexing）程序来同时监听多个套接字，并根据套接字目前执行的任务来为套接字关联不同的事件处理器。
- 当被监听的套接字准备好执行连接accept、read、write、close等操作时，与操作相对应的文件事件就会产生，这时文件事件处理器就会调用套接字之前关联好的事件处理器来处理这些事件。

虽然文件事件处理器以单线程方式运行，但通过使用I/O多路复用程序来监听多个套接字，文件事件处理器既实现了高性能的网络通信模型，又可以很好地与redis服务器中其他同样以单线程方式运行的模块进行对接，这保持了Redis内部单线程设计的简单性。

那 Redis6.0 之前为什么不使用多线程？我觉得主要原因有 3 点：

- 单线程编程容易并且更容易维护；
- Redis 的性能瓶颈不在 CPU，主要在内存和网络；
- 多线程就会存在死锁、线程上下文切换等问题，甚至会影响性能。

2.Redis6.0 之后为何引入了多线程？

Redis6.0 引入多线程主要是为了提高网络 IO 读写性能，因为这个算是 Redis 中的一个性能瓶颈（Redis 的瓶颈主要受限于内存和网络）。

虽然，Redis6.0 引入了多线程，但是 Redis 的多线程只是在网络数据的读写这类耗时操作上使用了，执行命令仍然是单线程顺序执行。因此，你也不需要担心线程安全问题。

[3]redis作缓存读写一致性问题

1.3种常用的缓存读写策略详解

1. Cache Aside Pattern（旁路缓存模式）

Cache Aside Pattern 是我们平时使用比较多的一个缓存读写模式，比较适合读请求比较多的场景。

Cache Aside Pattern 中服务端需要同时维系 db 和 cache，并且是以 db 的结果为准。

写：

- 先更新 db
- 然后直接删除 cache。

读：

- 从 cache 中读取数据，读取到就直接返回
- cache 中读取不到的话，就从 db 中读取数据返回
- 再把数据放到 cache 中。

1.在写数据的过程中，可以先删除 cache，后更新 db 么？

不行，因为这样可能会造成 **数据库（db）和缓存（Cache）数据不一致**的问题。举例：请求 1 先写数据 A，请求 2 随后读数据 A 的话，就很有可能产生数据不一致性的问题。

过程如下：请求 1 先把 cache 中的 A 数据删除 -> 请求 2 从 db 中读取数据->请求 1 再把 db 中的 A 数据更新。

2.在写数据的过程中，先更新 db，后删除 cache 就没有问题了么？

理论上来说还是可能会出现数据不一致性的问题，不过概率非常小，因为缓存的写入速度是比数据库的写入速度快很多。举例：请求 1 先读数据 A，请求 2 随后写数据 A，并且数据 A 在请求 1 请求之前不在缓存中的话，也有可能产生数据不一致性的问题。

大概过程是：请求 1 从 db 读数据 A-> 请求 2 更新 db 中的数据 A（此时缓存中无数据 A，故不用执行删除缓存操作）-> 请求 1 将数据 A 写入 cache

3.Cache Aside Pattern 的缺陷和解决办法

缺陷 1：首次请求数据一定不在 cache 的问题

解决办法：可以将热点数据可以提前放入 cache 中。

缺陷 2：写操作比较频繁的话导致 cache 中的数据会被频繁被删除，这样会影响缓存命中率。

解决办法：

- 数据库和缓存数据强一致场景：更新 db 的时候同样更新 cache，采用redisson实现的读写锁。
在读的时候添加共享锁，可以保证读读不互斥，读写互斥(其他线程可以一起读，但是不能写)。当我们更新数据的时候，添加排他锁，它是读写，读读都互斥（其他线程不能读也不能写），这样就能保证在写数据的同时是不会让其他线程读数据的，**避免了脏数据**。
那这个排他锁是如何保证读写、读读互斥的呢？
其实排他锁底层使用也是setnx，保证了同时只能有一个线程操作锁住的方法
- 可以短暂地允许数据库和缓存数据不一致的场景：更新 db 的时候同样更新 cache，但是给缓存加一个比较短的过期时间，这样的话就可以保证即使数据不一致的话影响也比较小。

2.Read/Write Through Pattern（读写穿透）

只更新缓存，由缓存自己同步更新数据库

写 (Write Through)：

- 先查 cache，cache 中不存在，直接更新 db。
- cache 中存在，则先更新 cache，然后 cache 服务自己更新 db（**同步更新 cache 和 db**）。

读(Read Through):

- 从 cache 中读取数据，读取到就直接返回。
- 读取不到的话，先从 db 加载，写入到 cache 后返回响应。

和 Cache Aside Pattern 一样，Read-Through Pattern 也有首次请求数据一定不再 cache 的问题，对于热点数据可以提前放入缓存中。

3.Write Behind Pattern（异步缓存写入）

只更新缓存，由缓存自己异步更新数据库

2.redis做为缓存，mysql的数据如何与redis进行同步呢？（双写一致性）

[认识MySQL和Redis的数据一致性问题-腾讯云开发者社区-腾讯云\(tencent.com\)](https://cloud.tencent.com/developer/article/1254444)

双写一致性：当修改了数据库的数据也要同时更新缓存的数据，缓存和数据库的数据要保持一致

Cache Aside Pattern (旁路缓存模式)中遇到写请求是这样的：更新 DB，然后直接删除 cache。

如果更新数据库成功，而删除缓存这一步失败的情况的话，简单说两个解决方案：

1. **缓存失效时间变短（不推荐，治标不治本）**：我们让缓存数据的过期时间变短，这样的话缓存就会从数据库中加载数据。另外，这种解决办法对于先操作缓存后操作数据库的场景不适用。
2. **增加 cache 更新重试机制（常用）**：如果 cache 服务当前不可用导致缓存删除失败的话，我们就隔一段时间进行重试，重试次数可以自己定。如果多次重试还是失败的话，我们可以把当前更新失败的 key 存入队列中，等缓存服务可用之后，再将缓存中对应的 key 删除即可。

怎么解决读写

1.若一致性要求高：强一致方案

采用redisson实现的读写锁。

在读的时候添加共享锁，可以保证读读不互斥，读写互斥(其他线程可以一起读，但是不能写)。当我们更新数据的时候，添加排他锁，它是读写，读读都互斥（其他线程不能读也不能写），这样就能保证在写数据的同时是不会让其他线程读数据的，**避免了脏数据**。

那这个排他锁是如何保证读写、读读互斥的呢？

其实排他锁底层使用也是setnx，保证了同时只能有一个线程操作锁住的方法

2.延时双删

- **策略原理**：延时双删策略的核心是在写库操作的前后分别进行删除缓存操作，并设定合理的超时时间来确保读请求结束，写请求可以删除可能产生的缓存脏数据。
- **具体步骤**：先删除缓存，再写数据库，然后线程休眠一段时间（比如500毫秒），最后再次删除缓存。休眠时间的确定需要评估项目读数据业务逻辑的耗时，并考虑Redis和数据库主从同步的耗时。
- **优缺点**：这种策略能在一定程度上解决数据不一致的问题，但增加了写请求的耗时，并且在最差的超时时间内，数据仍可能存在不一致性。

3.先写db再删除缓存策略

- **推荐理由**：这是实时性最好的方案，适用于对并发和实时一致性要求较高的场景。如果删除Redis失败，则可以通过重试机制来解决。
- **执行流程**：先执行数据库的更新操作，成功后立即删除Redis中的对应缓存。
- **优缺点**：虽然在极少数情况下会出现数据不一致的情况，但其条件相对苛刻，是实时性要求下的最佳选择。

[4]redis生产问题

1.缓存穿透，缓存雪崩，缓存击穿问题

1.什么是缓存穿透（数据库缓存都没有）？怎么解决？

缓存穿透:请求根本不存在的资源（DB本身就不存在，Redis更是不存在），这将导致这个不存在的数据每次请求都要到 DB 去查询，可能导致 DB 挂掉。这种情况大概率是遭到了攻击。

有2个解决方案:

方案一:使用布隆过滤器

使用BitMap作为布隆过滤器，将目前所有可以访问到的资源通过简单的映射关系放入到布隆过滤器中（哈希计算），当一个请求来临的时候先进行布隆过滤器的判断，如果有那么才进行放行，否则就直接拦截

布隆过滤器

什么是布隆过滤器？

布隆过滤器的核心思想是使用多个哈希函数来将元素映射到**位数组**中的多个位置上。当一个元素被加入到布隆过滤器中时，它会被多次哈希，并将对应的位数组位置设置为1。当需要判断一个元素是否在布隆过滤器中时，我们只需将该元素进行多次哈希，并检查对应的位数组位置是否都为1，如果其中有任意一位为0，则说明该元素不在集合中；如果所有位都为1，则说明该元素**可能**在集合中（因为有可能存在哈希冲突），需要进一步检查。

布隆过滤器主要是用于检索一个元素是否在一个集合中。我们当时使用的是redisson实现的布隆过滤器。它的底层主要是先去初始化一个比较大数组，里面存放的二进制0或1。在一开始都是0，当一个key来了之后经过3次hash计算，模于数组长度找到数据的下标然后把数组中原来的0改为1，这样的话，三个数组的位置就能表明一个key的存在。查找的过程也是一样的。但由于哈希冲突，存在一定的误判率，我们一般可以设置这个误判率，大概不会超过5%，其实这个误判是必然存在的，要不就得增加数组的长度，其实已经算是很划分了，5%以内的误判率一般的项目也能接受，不至于高并发下压倒数据库。

方案二:对空值进行缓存

如果查询缓存和数据库的数据没有找到，则直接设置一个默认值（可以是空值）存到缓存中，这样第二次读取缓存时就会获取到默认值，而不会继续访问数据库。

2.什么是缓存击穿（缓存没有数据库有）？怎么解决？

缓存击穿的意思是:redis某个热点key过期或者刚开始，但是此时有大量的用户访问该过期key（或者大并发场景下刚开始这个数据只在数据库里不在缓存里），这个时候大并发的请求可能会瞬间把DB压垮。

解决方案有两种方式：

第一可以使用互斥锁：只有一个请求可以获取到互斥锁，然后到DB中将数据查询并加入到缓存，之后所有请求就可以从缓存中得到响应

第二监控数据，实时调整:监控哪些数据是热门数据，实时的调整key的过期时长。

3.什么是缓存雪崩？怎么解决？

缓存雪崩是指:redis中大量的key集体过期,请求全部转发到DB，DB 瞬时压力过重雪崩。

与缓存击穿的区别：雪崩是很多key，击穿是某一个key缓存。

解决方式:将缓存失效时间分散开，比如可以在原有的失效时间基础上增加一个随机值，比如1-5分钟随机，这样每一个缓存的过期时间的重复率就会降低，就很难引发集体失效的事件

2.缓存预热

缓存预热就是系统上线后，将相关的缓存数据直接加载到缓存系统。这样就可以避免在用户请求的时候，先查询数据库，然后再将数据缓存的问题！用户直接查询事先被预热的缓存数据！

解决方案：

1. 直接写个缓存刷新页面，上线时手工操作一下；
2. 数据量不大，可以在项目启动的时候自动进行加载；
3. 定时刷新缓存；

3.缓存降级

当访问量剧增、服务出现问题（如响应时间慢或不响应）或非核心服务影响到核心流程的性能时，仍然需要保证服务还是可用的，即使是有损服务。系统可以根据一些关键数据进行自动降级，也可以配置开关实现人工降级。

服务降级的目的，是为了防止Redis服务故障，导致数据库跟着一起发生雪崩问题。因此，对于不重要的缓存数据，可以采取服务降级策略，例如一个比较常见的做法就是，Redis出现问题，不去数据库查询，而是直接返回默认值给用户。

4.redis阻塞问题

1.O(n) 命令

Redis 中的大部分命令都是 O(1)时间复杂度，但也有少部分 O(n) 时间复杂度的命令，例如：

- `KEYS *`：会返回所有符合规则的 key。
- `HGETALL`：会返回一个 Hash 中所有的键值对。
- `LRANGE`：会返回 List 中指定范围内的元素。
- `SMEMBERS`：返回 Set 中的所有元素。
- `SINTER / SUNION / SDIFF`：计算多个 Set 的交集/并集/差集。
-

redis是单线程的。keys指令会导致线程阻塞一段时间，直到执行完毕，服务才能恢复。

2.SAVE 创建 RDB 快照

Redis 提供了两个命令来生成 RDB 快照文件：

- `save`：同步保存操作，会阻塞 Redis 主线程；
- `bgsave`：fork 出一个子进程，子进程执行，不会阻塞 Redis 主线程，默认选项。

默认情况下，Redis 默认配置会使用 `bgsave` 命令。如果手动使用 `save` 命令生成 RDB 快照文件的话，就会阻塞主线程。

[5]redis内存管理

1.Redis 是如何判断数据是否过期的呢？

Redis 通过一个叫做过期字典（可以看作是 hash 表）来保存数据过期的时间。过期字典的键指向 Redis 数据库中的某个 key(键)，过期字典的值是一个 long long 类型的整数，这个整数保存了 key 所指向的数据库键的过期时间（毫秒精度的 UNIX 时间戳）。

2.Redis数据过期策略

(类似问题:假如redis的key过期之后，会立即删除吗？)

第一种是惰性删除，在设置该key过期时间后，我们不去管它，当需要该key时，我们在检查其是否过期，如果过期，我们就删掉它，反之返回该key。

第二种是 定期删除，就是说每隔一段时间抽取一批 key 执行删除过期 key 操作。并且，Redis 底层会通过限制删除操作执行的时长和频率来减少删除操作对 CPU 时间的影响。

定期删除对内存更加友好，惰性删除对 CPU 更加友好。两者各有千秋，所以 Redis 采用的是 **定期删除+惰性/懒汉式删除**。

但是，仅仅通过给 key 设置过期时间还是有问题的。因为还是可能存在定期删除和惰性删除漏掉了很多过期 key 的情况。这样就导致大量过期 key 堆积在内存里，然后就 Out of memory 了。

怎么解决这个问题呢？答案就是：**Redis 内存淘汰机制**。

3.Redis数据淘汰策略

(相关问题：MySQL 里有 2000w 数据，Redis 中只存 20w 的数据，如何保证 Redis 中的数据都是热点数据?)

数据的淘汰策略：当Redis中的内存不够用时，此时在向Redis中添加新的key，那么Redis就会按照某一种规则将内存中的数据删除掉，这种数据的删除规则被称之为内存的淘汰策略。

作为内存数据库，出于对性能和内存消耗的考虑，Redis 的淘汰算法实际实现上并非针对所有 key，而是抽样一小部分并且从中选出被淘汰的 key。

这个在redis中提供了很多种,默认是noeviction，不删除任何数据，内存不足直接报错。

Redis 提供 6 种数据淘汰策略：

1. **volatile-lru (least recently used)**：从已设置过期时间的数据集 (`server.db[i].expires`) 中挑选最近最少使用的数据淘汰。
2. **volatile-ttl**：从已设置过期时间的数据集 (`server.db[i].expires`) 中挑选将要过期的数据淘汰。
3. **volatile-random**：从已设置过期时间的数据集 (`server.db[i].expires`) 中任意选择数据淘汰。
4. **allkeys-lru (least recently used)**：当内存不足以容纳新写入数据时，在键空间中，移除最近最少使用的 key（这个是最常用的）。
5. **allkeys-random**：从数据集 (`server.db[i].dict`) 中任意选择数据淘汰。
6. **no-eviction**：禁止驱逐数据，也就是说当内存不足以容纳新写入数据时，新写入操作会报错。这个应该没人使用吧！

4.0 版本后增加以下两种：

1. **volatile-lfu (least frequently used)**：从已设置过期时间的数据集 (`server.db[i].expires`) 中挑选最不经常使用的数据淘汰。
2. **allkeys-lfu (least frequently used)**：当内存不足以容纳新写入数据时，在键空间中，移除最不经常使用的 key。

LRU和LFU

这是可以在redis的配置文件中设置，里面有两个非常重要的概念，一个是LRU，另外一个为LFU

LRU(**least recently used**)的意思就是最少最近使用，用当前时间减去最后一次访问时间，这个值越大则淘汰优先级越高。

LFU(**least frequently used**)的意思是最少频率使用。会统计每个key的访问频率，值越小淘汰优先级越高

使用 Redis 缓存数据时，为了提高缓存命中率，需要保证缓存数据都是热点数据。可以将内存最大使用量设置为热点数据占用的内存量，然后启用 allkeys-lru 淘汰策略，将最近最少使用的数据淘汰，把一些经常访问的key留在redis中。

[6].Redis持久化机制

Redis 是内存型数据库，为了保证数据在断电后不会丢失，需要将内存中的数据持久化到硬盘上

1.RDB和AOF的对比

Redis支持两种方式的持久化，一种是RDB的方式，一种是AOF的方式。前者会根据指定的规则定时将内存中的数据存储在硬盘上，而后者在每次执行完命令后将命令记录下来。一般将两者结合使用

	RDB	AOF
持久化方式	定时对整个内存做快照	记录每一次执行的命令
数据完整性	不完整，两次备份之间会丢失	相对完整，取决于刷盘策略
文件大小	会有压缩，文件体积小	记录命令，文件体积很大
宕机恢复速度	很快	慢
数据恢复优先级	低，因为数据完整性不如AOF	高，因为数据完整性更高
系统资源占用	高，大量CPU和内存消耗	低，主要是磁盘IO资源 但AOF重写时会占用大量CPU和内存资源
使用场景	可以容忍数分钟的数据丢失，追求更快的启动速度	对数据安全性要求较高常见

- RDB(snapshotting，快照)是一个快照文件，定时对整个数据做快照，把redis内存存储的数据写到磁盘上，当redis实例宕机恢复数据的时候，方便从RDB的快照文件中恢复数据。RDB方法通过fork创建子进程来进行实际的快照保存工作，这样可以最小化对主线程的影响。当满足一定条件时，例如在指定的时间间隔内数据集发生了指定次数的更改，Redis会自动执行bgsave命令来生成RDB文件。此外，也可以通过手动执行save或bgsave命令来触发RDB快照的生成。
- AOF(append-only file,只追加文件)的含义是追加文件，在AOF模式下，Redis将所有修改数据的操作以文本形式记录到一个日志文件中，即每条写命令都被附加到文件的末尾。这种持久化方法能够更好地保证数据完整性，因为即使出现系统崩溃，也可以从AOF文件中重现所有的写操作来恢复数据。

2.这两种方式的优缺点和使用场景？

RDB和AOF两种持久化方式各有优劣。RDB文件是一个压缩的二进制文件，占用空间较小，恢复速度快，但可能会丢失最近一次快照之后的数据更新。而AOF以日志的形式记录写操作，带来了数据更完整性的保证，但由于是文本格式，其文件体积较大，恢复速度也相对较慢。

通常来说，应该同时使用两种持久化方案，以保证数据安全。

- 如果数据不敏感，且可以从其他地方重新生成，可以关闭持久化。
- 如果数据比较重要，且能够承受几分钟的数据丢失，比如缓存等，只需要使用RDB即可。
- 如果是用做内存数据，要使用Redis的持久化，建议是RDB和AOF都开启。
- 如果只用AOF，优先使用everysec的配置选择，因为它在可靠性和性能之间取了一个平衡。

当RDB与AOF两种方式都开启时，Redis会优先使用AOF恢复数据，因为AOF保存的文件比RDB文件更完整。

2.RDB快照机制

RDB(snapshotting，快照)是一个快照文件，定时对整个数据做快照，把redis内存存储的数据写到磁盘上，当redis实例宕机恢复数据的时候，方便从RDB的快照文件中恢复数据。如果数据量很大，保存快照的时间会很长。

RDB 创建快照时会阻塞主线程吗？

Redis 提供了两个命令来生成 RDB 快照文件：

- `save`：同步保存操作，会阻塞 Redis 主线程；
- `bgsave`：fork 出一个子进程，子进程执行，不会阻塞 Redis 主线程，默认选项。

当执行**bgsave**命令时，Redis服务器会立即返回OK响应，表明命令已经开始执行。随后，Redis会fork出一个子进程来负责实际的数据保存工作，而主进程则继续正常处理来自客户端的请求。这种方式既确保了数据的持久化，又保证了服务器的性能不受影响。

3.AOF只追加文件持久化机制

开启 AOF 持久化后每执行一条会更改 Redis 中的数据命令，Redis 就会将该命令写入到 AOF 缓冲区 `server.aof_buf` 中，然后再写入到 AOF 文件中（此时还在系统内核缓存区未同步到磁盘），最后再根据持久化方式（`fsync` 策略）的配置来决定何时将系统内核缓存区的数据同步到硬盘中的。

1.AOF 工作基本流程是怎样的？

AOF 持久化功能的实现可以简单分为 5 步：

1. **命令追加 (append)**：所有的写命令会追加到 AOF 缓冲区中：所有写入命令会先追加到缓冲区（`aof_buf`）中，而不是直接写入文件。这是为了减少磁盘IO操作，提升性能。Redis使用单线程响应命令，如果每次写操作都直接追加到硬盘，性能会受到严重影响。因此，先将命令写入缓冲区是一种折中方式。
2. **文件同步 (fsync)**：AOF 缓冲区根据对应的持久化方式（`fsync` 策略）向AOF文件做同步操作。这一步需要调用 `fsync` 函数（系统调用），`fsync` 针对单个文件操作，对其进行强制硬盘同步，`fsync` 将阻塞直到写入磁盘完成后返回，保证了数据持久化。（在AOF文件中，每一个写命令都会被记录下来，其中包括对键值的修改、添加和删除操作。然而，某些键可能会被多次修改或删除，这就造成了数据的冗余。在AOF重写过程中，只记录最终的结果，从而大大减少了文件大小。）
3. **文件重写 (rewrite)**：随着 AOF 文件越来越大，需要定期对 AOF 文件进行重写，达到压缩的目的。随着不断写入AOF文件，其体积会不断增大。为了减小文件体积并提高恢复速度，Redis提供了AOF文件重写机制。Redis通过fork创建一个子进程来进行重写，子进程根据RDB内存快照将AOF文件数据写入新的AOF文件，父进程继续接收并缓存新的写命令，同时将这些新的命令通过增量写入新的AOF文件。完成后，新AOF文件替换旧文件。
4. **重启加载 (load)**：当 Redis 重启时，可以加载 AOF 文件进行数据恢复。

2.AOF 持久化方式有哪些？

fsync是一个在Linux操作系统中非常重要的函数，用于将缓冲区中的数据立即写入磁盘，从而确保数据的持久性和安全性。

策略有三种：**always**、**everysec**和**no**。**always**表示每次写操作都会立即同步写入AOF文件，**everysec**表示每秒执行一次同步操作，**no**表示由操作系统决定何时进行同步。

在**always**策略下，命令写入缓冲区后，直接调用系统的**fsync**操作同步到AOF文件。在**everysec**策略下，命令写入缓冲区后，每秒执行一次**fsync**操作。而在**no**策略下，仅依靠操作系统的写入操作，不进行**fsync**同步。

在 Redis 的配置文件中存在三种不同的 AOF 持久化方式（`fsync` 策略），它们分别是：

1. `appendfsync always`：主线程调用 `write` 执行写操作后，后台线程（`aof_fsync` 线程）立即会调用 `fsync` 函数同步 AOF 文件（刷盘），`fsync` 完成后线程返回，这样会严重降低 Redis 的

性能 (`write` + `fsync`) 。

2. `appendfsync everysec`：主线程调用 `write` 执行写操作后立即返回，由后台线程 (`aof_fsync` 线程) 每秒钟调用 `fsync` 函数 (系统调用) 同步一次 AOF 文件 (`write` + `fsync` , `fsync` 间隔为 1 秒)
3. `appendfsync no`：主线程调用 `write` 执行写操作后立即返回，让操作系统决定何时进行同步，Linux 下一般为 30 秒一次 (`write` 但不 `fsync` , `fsync` 的时机由操作系统决定) 。

可以看出：这 3 种持久化方式的主要区别在于 `fsync` 同步 AOF 文件的时机 (刷盘) 。

3.AOF 为什么是在执行完命令之后记录日志?

关系型数据库 (如 MySQL) 通常都是执行命令之前记录日志 (方便故障恢复) , 而 Redis AOF 持久化机制是在执行完命令之后再记录日志。

为什么是在执行完命令之后记录日志呢?

- 避免额外的检查开销, AOF 记录日志不会对命令进行语法检查;
- 在命令执行完之后再记录, 不会阻塞当前的命令执行。

这样也带来了风险 (我在前面介绍 AOF 持久化的时候也提到过) :

- 如果刚执行完命令 Redis 就宕机会导致对应的修改丢失;
- 可能会阻塞后续其他命令的执行 (AOF 记录日志是在 Redis 主线程中进行的) 。

[7]redis事务

1.什么是 Redis 事务?

你可以将 Redis 中的事务理解为：**Redis 事务提供了一种将多个命令请求打包的功能。然后，再按顺序执行打包的所有命令，并且不会被中途打断。**

redis事务不能回滚，不满足原子性和持久性。

我们知道事务具有四大特性：**1. 原子性，2. 隔离性，3. 持久性，4. 一致性。**

1. **原子性 (Atomicity)**：事务是最小的执行单位，不允许分割。事务的原子性确保动作要么全部完成，要么完全不起作用；
2. **隔离性 (Isolation)**：并发访问数据库时，一个用户的事务不被其他事务所干扰，各并发事务之间数据库是独立的；
3. **持久性 (Durability)**：一个事务被提交之后。它对数据库中数据的改变是持久的，即使数据库发生故障也不应该对其有任何影响。
4. **一致性 (Consistency)**：执行事务前后，数据保持一致，多个事务对同一个数据读取的结果是相同的；

2. 如何使用 Redis 事务?

Redis 可以通过 `MULTI` , `EXEC` , `DISCARD` 和 `WATCH` 等命令来实现事务(Transaction)功能

MULTI命令后可以输入多个命令，Redis 不会立即执行这些命令，而是将它们放到队列，当调用了 **EXEC**命令后，再执行所有的命令。

这个过程是这样的：

1. 开始事务 (`MULTI`) ；

2. 命令入队(批量操作 Redis 的命令, 先进先出 (FIFO) 的顺序执行);
3. 执行事务(EXEC)。

DISCARD命令取消一个事务, 它会清空事务队列中保存的所有命令。

WATCH命令监听指定的 Key, 当调用 **EXEC** 命令执行事务时, 如果一个被 **WATCH** 命令监视的 Key 被其他客户端/Session 修改的话, 整个事务都不会被执行。

不过, 如果 **WATCH** 与 **事务** 在同一个 Session 里, 并且被 **WATCH** 监视的 Key 被修改的操作发生在事务内部, 这个事务是可以被执行成功的。

3.Redis 事务为什么不支持原子性?

Redis 的事务和我们平时理解的关系型数据库的事务不同。Redis 事务在运行错误的情况下, 除了执行过程中出现错误的命令外, 其他命令都能正常执行。并且, Redis 事务是不支持回滚 (roll back) 操作的。因此, Redis 事务其实是不满足原子性的。

Redis 官网也解释了自己为啥不支持回滚。简单来说就是 Redis 开发者们觉得没必要支持回滚, 这样更简单便捷并且性能更好。Redis 开发者觉得即使命令执行错误也应该在开发过程中就被发现而不是生产过程中。

4.Redis 事务为什么不支持持久性?

Redis 不同于 Memcached 的很重要一点就是, Redis 支持持久化, 而且支持 3 种持久化方式:

- 快照 (snapshotting, RDB)
- 只追加文件 (append-only file, AOF)
- RDB 和 AOF 的混合持久化(Redis 4.0 新增)

与 RDB 持久化相比, AOF 持久化的实时性更好。在 Redis 的配置文件中存在三种不同的 AOF 持久化方式 (**fsync** 策略) , 它们分别是:

```
appendfsync always    #每次有数据修改发生时都会调用fsync函数同步AOF文件,fsync完成后线程返回,这样会严重降低Redis的速度
appendfsync everysec   #每秒钟调用fsync函数同步一次AOF文件
appendfsync no         #让操作系统决定何时进行同步,一般为30秒一次
```

AOF 持久化的 **fsync** 策略为 **no**、**everysec** 时都会存在数据丢失的情况。 **always** 下可以基本是可以满足持久性要求的, 但性能太差, 实际开发过程中不会使用。

因此, Redis 事务的持久性也是没办法保证的。

5.如何解决 Redis 事务的缺陷?

我们可以利用 Lua 脚本来批量执行多条 Redis 命令, 这些 Redis 命令会被提交到 Redis 服务器一次性执行完成, 大幅减小了网络开销。

一段 Lua 脚本可以视作一条命令执行, 一段 Lua 脚本执行过程中不会有其他脚本或 Redis 命令同时执行, 保证了操作不会被其他指令插入或打扰。

不过, 如果 Lua 脚本运行时出错并中途结束, 出错之后的命令是不会被执行的。并且, 出错之前执行的命令是无法被撤销的, 无法实现类似关系型数据库执行失败可以回滚的那种原子性效果。因此, 严格来说的话, 通过 Lua 脚本来批量执行 Redis 命令实际也是不完全满足原子性的。

6.Redis事务支持隔离性吗?

Redis 是单进程程序，并且它保证在执行事务时，不会对事务进行中断，事务可以运行直到执行完所有事务队列中的命令为止。因此，Redis 的事务是总是带有隔离性的。

[8]redis性能优化

0.使用批量操作减少网络传输

一个 Redis 命令的执行可以简化为以下 4 步：

1. 发送命令
2. 命令排队
3. 命令执行
4. 返回结果

其中，第 1 步和第 4 步耗费时间之和称为 **Round Trip Time (RTT,往返时间)**，也就是数据在网络上传输的时间。

使用批量操作可以减少网络传输次数，进而有效减小网络开销，大幅减少 RTT。

1.原生批量操作命令

Redis 中有一些原生支持批量操作的命令，比如：

- **MGET** (获取一个或多个指定 key 的值)、**MSET** (设置一个或多个指定 key 的值)、
- **HMGET** (获取指定哈希表中一个或者多个指定字段的值)、**HMSET** (同时将一个或多个 field-value 对设置到指定哈希表中)、
- **SADD** (向指定集合添加一个或多个元素)
-

不过，在 Redis 官方提供的分片集群解决方案 Redis Cluster 下，使用这些原生批量操作命令可能会存在一些小问题需要解决。就比如说 **MGET** 无法保证所有的 key 都在同一个 **hash slot** (哈希槽) 上，**MGET** 可能还是需要多次网络传输，原子操作也无法保证了。不过，相较于非批量操作，还是可以节省不少网络传输次数。

整个步骤的简化版如下（通常由 Redis 客户端实现，无需我们自己再手动实现）：

1. 找到 key 对应的所有 hash slot(哈希槽)；
2. 分别向对应的 Redis 节点发起 **MGET** 请求获取数据；
3. 等待所有请求执行结束，重新组装结果数据，保持跟入参 key 的顺序一致，然后返回结果。

2.pipeline管道

对于不支持批量操作的命令，我们可以利用 **pipeline (管道)** 将一批 Redis 命令封装成一组，这些 Redis 命令会被一次性提交到 Redis 服务器，只需要一次网络传输。不过，需要注意控制一次批量操作的 **元素个数**(例如 500 以内，实际也和元素字节数有关)，避免网络传输的数据量过大。

使用 **pipeline** 组装的命令个数不能太多，不然数据量过大，增加客户端的等待时间，还可能造成网络阻塞，可以将大量命令的拆分多个小的 **pipeline** 命令完成。

工作原理

当使用Pipeline时，客户端会将多个命令缓存起来形成一个命令序列，然后一次性发送给服务器。服务器端对这些命令进行顺序处理，并最终将所有命令的结果一次性返回。这种机制显著减少了因多次网络请求带来的额外延迟。

3.原生批命令（mset和mget）与 pipeline 对比

原生批命令（mset和mget）与 pipeline 对比：

1. **原生批量操作命令是原子操作，pipeline 是非原子操作**（如果一个Pipeline中的命令在执行过程中出现故障，之前已经执行的命令不会回滚，它们的结果仍然有效。）。
2. pipeline 可以打包不同的命令，原生批量操作命令不可以。
3. 原生批量操作命令是 Redis 服务端支持实现的，而 pipeline 需要服务端和客户端的共同实现。

顺带补充一下 pipeline 和 Redis 事务的对比：

- 事务是原子操作，pipeline 是非原子操作。两个不同的事务不会同时运行，而 pipeline 可以同时以交错方式执行。
- Redis 事务中每个命令都需要发送到服务端，而 Pipeline 只需要发送一次，请求次数更少。

1.redis bigkey（大 Key）

什么是 bigkey？

简单来说，如果一个 key 对应的 value 所占用的内存比较大，那这个 key 就可以看作是 bigkey。

bigkey 是怎么产生的？有什么危害？

bigkey 通常是由于下面这些原因产生的：

- 程序设计不当，比如直接使用 String 类型存储较大的文件对应的二进制数据。
- 对于业务的数据规模考虑不周到，比如使用集合类型的时候没有考虑到数据量的快速增长。
- 未及时清理垃圾数据，比如哈希中冗余了大量的无用键值对。

bigkey 除了会消耗更多的内存空间和带宽，还会对性能造成比较大的影响。

如何发现 bigkey？

使用 Redis 自带的 SCAN 命令

SCAN 命令可以按照一定的模式和数量返回匹配的 key。获取了 key 之后，可以利用 STRLEN、HLEN、LLEN 等命令返回其长度或成员数量。

如何处理 bigkey？

bigkey 的常见处理以及优化办法如下（这些方法可以配合起来使用）：

- **拆分BigKey**
 - **对于String类型的BigKey**：如果String类型的值超过10MB，建议将其拆分为多个较小的字符串键值对。这样，每个键值对的大小更合理，便于管理。
 - **对于复杂数据类型的BigKey**：如Set、List或Hash类型，当成员数量过多时（比如10000个成员），可以将这些集合拆分成多个小集合。例如，一个包含大量成员的大Hash可以按某种业务规则分割成若干个小Hash。这种拆分既可以减少单个Key的数据量，也有助于后续的数据维护和查询。
- **手动清理**：Redis 4.0+ 可以使用 UNLINK 命令来异步删除一个或多个指定的 key。Redis 4.0 以下可以考虑使用 SCAN 命令结合 DEL 命令来分批次删除。
- **采用合适的数据结构**：例如，文件二进制数据不使用 String 保存、使用 HyperLogLog 统计页面 UV、Bitmap 保存状态信息（0/1）。

- **开启 lazy-free (惰性删除/延迟释放)** : lazy-free 特性是 Redis 4.0 开始引入的, 指的是让 Redis 采用异步方式延迟释放 key 使用的内存, 将该操作交给单独的子线程处理, 避免阻塞主线程。

2.redis hotkey (热 Key)

什么是 hotkey?

如果一个 key 的访问次数比较多且明显多于其他 key 的话, 那这个 key 就可以看作是 **hotkey (热 Key)**。例如在 Redis 实例的每秒处理请求达到 5000 次, 而其中某个 key 的每秒访问量就高达 2000 次, 那这个 key 就可以看作是 hotkey。

hotkey 出现的原因主要是某个热点数据访问量暴增, 如重大的热搜事件、参与秒杀的商品。

hotkey 有什么危害?

处理 hotkey 会占用大量的 CPU 和带宽, 可能会影响 Redis 实例对其他请求的正常处理。此外, 如果突然访问 hotkey 的请求超出了 Redis 的处理能力, Redis 就会直接宕机。这种情况下, 大量请求将落到后面的数据库上, 可能会导致数据库崩溃。

因此, hotkey 很可能成为系统性能的瓶颈点, 需要单独对其进行优化, 以确保系统的高可用性和稳定性。

如何发现 hotkey?

使用 Redis 自带的 `--hotkeys` 参数来查找。

Redis 4.0.3 版本中新增了 `hotkeys` 参数, 该参数能够返回所有 key 的被访问次数。

如何解决 hotkey?

hotkey 的常见处理以及优化办法如下 (这些方法可以配合起来使用) :

- **读写分离**: 主节点处理写请求, 从节点处理读请求。
- **使用 Redis Cluster**: 将热点数据分散存储在多个 Redis 节点上。
- **二级缓存**: hotkey 采用二级缓存的方式进行处理, 将 hotkey 存放一份到 JVM 本地内存中 (可以用 Caffeine) 。

3.redis内存碎片

1.什么是内存碎片?

你可以将内存碎片简单地理解为那些不可用的空闲内存。

2.Redis 内存碎片产生比较常见的 2 个原因:

1、Redis 存储存储数据的时候向操作系统申请的内存空间可能会大于数据实际需要的存储空间。

Redis 使用 `zmalloc` 方法(Redis 自己实现的内存分配方法)进行内存分配的时候, 除了要分配 `size` 大小的内存之外, 还会多分配 `PREFIX_SIZE` 大小的内存。

另外, Redis 可以使用多种内存分配器来分配内存 (libc、jemalloc、tcmalloc), 默认使用 jemalloc, 而 jemalloc 按照一系列固定的大小 (8 字节、16 字节、32 字节.....) 来分配内存的。

当程序申请的内存最接近某个固定值时, jemalloc 会给它分配相应大小的空间, 比如说程序需要申请 17 字节的内存, jemalloc 会直接给它分配 32 字节的内存, 这样会导致有 15 字节内存的浪费。不过, jemalloc 专门针对内存碎片问题做了优化, 一般不会存在过度碎片化的问题。

2、频繁修改 Redis 中的数据也会产生内存碎片。

当 Redis 中的某个数据删除时，Redis 通常不会轻易释放内存给操作系统。

3.如何查看 Redis 内存碎片的信息？

使用 `info memory` 命令即可查看 Redis 内存相关的信息。

4.如何清理 Redis 内存碎片？

Redis4.0-RC3 版本以后自带了内存整理，可以避免内存碎片率过大的问题。

直接通过 `config set` 命令将 `activedefrag` 配置项设置为 `yes` 即可。

[9]Redis分布式锁问题

1.为什么需要分布式锁？

在多线程环境中，如果多个线程同时访问共享资源（例如商品库存、外卖订单），会发生数据竞争，可能会导致出现脏数据或者系统问题，威胁到程序的正常运行。

举个例子，假设现在有 100 个用户参与某个限时秒杀活动，每位用户限购 1 件商品，且商品的数量只有 3 个。如果不对共享资源进行互斥访问，就可能出现以下情况：

- 线程 1、2、3 等多个线程同时进入抢购方法，每一个线程对应一个用户。
- 线程 1 查询用户已经抢购的数量，发现当前用户尚未抢购且商品库存还有 1 个，因此认为可以继续执行抢购流程。
- 线程 2 也执行查询用户已经抢购的数量，发现当前用户尚未抢购且商品库存还有 1 个，因此认为可以继续执行抢购流程。
- 线程 1 继续执行，将库存数量减少 1 个，然后返回成功。
- 线程 2 继续执行，将库存数量减少 1 个，然后返回成功。
- 此时就发生了超卖问题，导致商品被多卖了一份。

为了保证共享资源被安全地访问，我们需要使用互斥操作对共享资源进行保护，即同一时刻只允许一个线程访问共享资源，其他线程需要等待当前线程释放后才能访问。这样可以避免数据竞争和脏数据问题，保证程序的正确性和稳定性。

如何才能实现共享资源的互斥访问呢？锁是一个比较通用的解决方案，更准确点来说是悲观锁。

悲观锁总是假设最坏的情况，认为共享资源每次被访问的时候就会出现问题(比如共享数据被修改)，所以每次在获取资源操作的时候都会上锁，这样其他线程想拿到这个资源就会阻塞直到锁被上一个持有者释放。也就是说，**共享资源每次只给一个线程使用，其它线程阻塞，用完后再把资源转让给其它线程。**

对于单机多线程来说，在 Java 中，我们通常使用 `ReentrantLock` 类、`synchronized` 关键字这类 JDK 自带的 **本地锁** 来控制一个 JVM 进程内的多个线程对本地共享资源的访问。

这些线程访问共享资源是互斥的，同一时刻只有一个线程可以获取到本地锁访问共享资源。

分布式系统下，不同的服务/客户端通常运行在独立的 JVM 进程上。如果多个 JVM 进程共享同一份资源的话，使用本地锁就没办法实现资源的互斥访问了。于是，**分布式锁** 就诞生了。

举个例子：系统的订单服务一共部署了 3 份，都对外提供服务。用户下订单之前需要检查库存，为了防止超卖，这里需要加锁以实现检查库存操作的同步访问。由于订单服务位于不同的 JVM 进程中，本地锁在这种情况下就没办法正常工作了。我们需要用到分布式锁，这样的话，即使多个线程不在同一个 JVM 进程中也能获取到同一把锁，进而实现共享资源的互斥访问。

2.redis的分布式锁redission的工作原理

redission底层是redis的setnx命令和lua脚本。

他的工作过程如下：

一个线程加锁成功后，会另开一个线程（称为看门狗Watch dog）进行监控，不断监听持有锁的线程，给线程增加持有锁的时间，也就是“续期”，规则是每隔releaseTime(锁的过期时间)/3的时间做一次续期（就是重新设置锁过期时间为releaseTime），手动释放锁，此时通知对应线程的Watch dog不需要再监听了。

另外一个线程也想加锁,while循环不断尝试获取锁，到了一定时间与之后就会放弃。

redis本身怎么实现锁？

- **SETEX** key seconds value 设置指定key的值，并将 key 的过期时间设为 seconds 秒
- **SETNX** key value 只有在 key 不存在时设置 key 的值

3.redission实现的分布式锁可重入吗

可重入就是说某个线程已经获得某个锁，该线程可以再次获取锁而不会出现死锁。

可重入锁，也叫做递归锁，指的是在同一线程内，外层函数获得锁之后，内层递归函数仍然可以获取到该锁。说白了就是同一个线程再次进入同样代码时，可以再次拿到该锁。它的作用是：防止在同一线程中多次获取锁而导致死锁发生。

redisson实现的分布式锁是可以重入的。这样做是为了避免死锁的产生。这个重入其实在内部就是判断是否是当前线程持有的锁，如果是当前线程持有的锁就会计数，如果释放锁就会在计算上减一。在存储数据的时候采用的hash结构，其中key是当前线程的唯一标识，value是当前线程重入的次数。

4.redisson实现的分布式锁能解决主从一致性的问题吗

redisson实现的分布式锁不能解决主从一致性的问题，比如，当线程1加锁成功后，master节点数据会异步复制到slave节点，此时当前持有Redis锁的master节点宕机，slave节点被提升为新的master节点，假如现在来了一个线程2，再次加锁，会在新的master节点上加锁成功，这个时候就会出现**两个节点同时持有一把锁**的问题。

我们可以利用redisson提供的**红锁**来解决这个问题，它的主要作用是，**不能只在一个redis实例上创建锁，应该是在多个redis实例上创建锁，并且要求在大多数redis节点上都成功创建锁**，红锁中要求是redis的节点数量要过半。这样就能避免线程1加锁成功后master节点宕机导致线程2成功加锁到新的master节点上的问题了。

但是，如果使用了红锁，因为需要同时都在多个节点上都添加锁，性能就变的很低了，并且运维维护成本也非常高，所以，我们一般在项目中也不会直接使用红锁，并且官方也暂时废弃了这个红锁

如果业务非要保证数据的强一致性，建议使用zookeeper实现的分布式锁，它是可以保证强一致性的。

[10]Redis集群方案

1.redis集群方案有哪些

- 主从复制
- 哨兵模式
- 分片集群

2.主从复制(解决高并发问题)及其原理/流程

单节点Redis的并发能力是有上限的，要进一步提高Redis的并发能力，可以搭建主从集群，实现**读写分离**。一般都是一主多从，主节点负责写数据，从节点负责读数据，主节点写入数据之后，需要把数据同步到从节点中

1.主从复制作用

- 1.数据冗余：主从复制实现了数据的热备份，是持久化之外的一种数据冗余方式。
- 2.故障恢复：当主节点出现问题时，可以由从节点提供服务，实现快速的故障恢复；实际上是一种服务的冗余。
- 3.负载均衡：**在主从复制的基础上，配合读写分离，可以由主节点提供写服务，由从节点提供读服务（即写Redis数据时应用连接主节点，读Redis数据时应用连接从节点），分担服务器负载；尤其是在写少读多的场景下，通过多个从节点分担读负载，可以大大提高Redis服务器的并发量。**
- 4.高可用基石：除了上述作用以外，主从复制还是哨兵和集群能够实施的基础，因此说主从复制是Redis高可用的基础。

2.主从同步流程

主从同步分为了两个阶段，一个是全量同步，一个是增量同步

全量同步是指**从节点第一次与主节点建立连接的时候使用全量同步**，流程是这样的：

第一：从节点请求主节点同步数据，其中从节点会携带自己的replication id和offset偏移量。

第二：主节点**判断是否是第一次请求**，主要判断的依据就是，主节点与从节点是否是同一个replication id，如果不是，就说明是第一次同步，那主节点就会把自己的replication id和offset发送给从节点，让从节点与主节点的信息保持一致。

第三：在同时主节点会**执行bgsave，生成rdb文件后，发送给从节点去执行**，从节点先把自己的数据清空，然后执行主节点发送过来的rdb文件，这样就保持了一致

当然，如果在rdb生成执行期间，依然有请求到了主节点，而主节点会以命令的方式(AOF方式)记录到一个日志文件，最后把这个日志文件发送给从节点，这样就能保证主节点与从节点完全一致了，后期再同步数据的时候，都是依赖于这个日志文件，这个就是全量同步

增量同步指的是，当从节点服务重启之后，数据就不一致了，所以这个时候，从节点会请求主节点同步数据，主节点还是判断不是第一次请求，**不是第一次就获取从节点的offset值，然后主节点从AOF命令日志中获取offset值之后的数据，发送给从节点进行数据同步**

3.哨兵模式（解决高可用问题）

主从复制存在不能自动故障转移、达不到高可用的问题。哨兵模式解决了这些问题。通过哨兵机制可以自动切换主从节点。

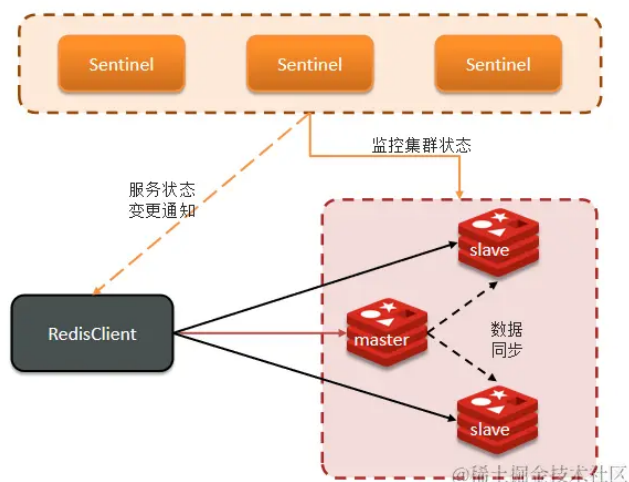
客户端连接Redis的时候，先连接哨兵，哨兵会告诉客户端Redis主节点的地址，然后客户端连接上Redis并进行后续的操作。当主节点宕机的时候，哨兵监测到主节点宕机，会重新推选出某个表现良好的从节点成为新的主节点，然后通过发布订阅模式通知其他的从服务器，让它们切换主机。

1.哨兵的作用和工作原理(心跳机制，选主规则)

哨兵的作用

Redis提供了哨兵（Sentinel）机制来实现主从集群的自动故障恢复。哨兵的结构和作用如下：

- **监控：***Sentinel* 会不断检查您的*master*和*slave*是否按预期工作
- **自动故障恢复：**如果*master*故障，*Sentinel*会将一个*slave*提升为*master*。当故障实例恢复后也以新的*master*为主
- **通知：***Sentinel*充当Redis客户端的服务发现来源，当集群发生故障转移时，会将最新信息推送给Redis的客户端



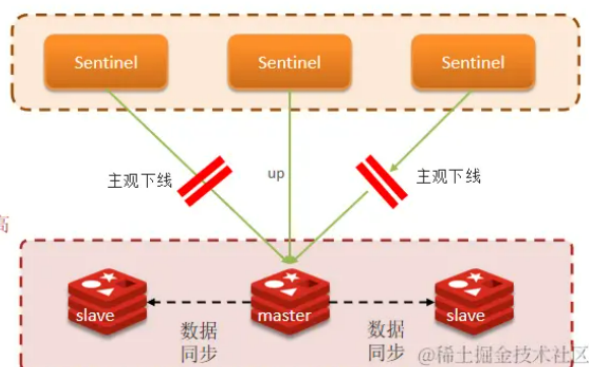
服务状态监控

*Sentinel*基于心跳机制监测服务状态，每隔1秒向集群的每个实例发送ping命令：

- **主观下线：**如果某*sentinel*节点发现某实例未在规定时间内响应，则认为该实例主观下线。
- **客观下线：**若超过指定数量（*quorum*）的*sentinel*都认为该实例主观下线，则该实例客观下线。*quorum*值最好超过*Sentinel*实例数量的一半。

哨兵选主规则

- 首先判断主与从节点断开时间长短，如超过指定值就排该从节点
- 然后判断从节点的*slave-priority*值，越小优先级越高
- 如果*slave-priority*一样，则判断*slave*节点的*offset*值，越大优先级越高
- 最后是判断*slave*节点的运行id大小，越小优先级越高。



9. 下面不属于Redis 中哨兵功能的是（）

- A 哨兵会不断地检查主节点和从节点是否正常工作
- B 当主节点不能正常工作时，哨兵实现主节点的自动故障转移
- C 哨兵可以将数据分散到多个节点**
- D 哨兵可以将故障转移的结果发送给客户端

正确答案：C

2.怎么保证Redis的高并发高可用

首先可以搭建主从集群，再加上使用redis中的哨兵模式，哨兵模式可以实现主从集群的自动故障恢复，里面就包含了对主从服务的监控、自动故障恢复、通知；如果master故障，Sentinel会将一个slave提升为master。当故障实例恢复后也以新的master为主；同时Sentinel也充当Redis客户端的服务发现来源，当集群发生故障转移时，会将最新信息推送给Redis的客户端，所以一般项目都会采用哨兵的模式来保证redis的高并发高可用

3.redis集群脑裂问题

一个Redis集群因为网络故障被分割成两部分，其中一部分包含主节点（Master），另一部分则由从节点（Slave）组成。如果此时主节点仍然能够接受客户端的写请求，而从节点因为与主节点失联而被哨兵系统提升为新的主节点，那么同一个数据就可能在两个“主节点”上被不同客户端写入，导致数据严重不一致。就像大脑分裂了一样，这样会导致客户端还在old master那里写入数据，新节点无法同步数据，当网络恢复后，sentinel会将old master降为slave，这时再从新master同步数据，这会导致old master中的大量数据丢失。

关于解决的话，我记得在redis的配置中可以设置：第一可以设置最少的slave节点个数，比如设置至少要有两个从节点才能同步数据，第二个可以设置主从数据复制和同步的延迟时间，达不到要求就拒绝请求，就可以避免大量的数据丢失

4.分片集群Redis cluster

哨兵模式解决了主从复制不能自动故障转移、达不到高可用的问题，但还是存在主节点的写能力、容量受限于单机配置的问题。而cluster模式实现了Redis的分布式存储，每个节点存储不同的内容，解决主节点的写能力、容量受限于单机配置的问题。

Redis的分片集群是一个分布式系统，用于将Redis的数据分布在多个Redis节点上，通过数据分片、哈希槽和去中心化的设计，实现了高可用性、高性能和易于水平扩展的特性。

Redis分片集群主要着眼于解决海量数据存储和高并发写的问题。在分片集群中，每个master节点负责保存不同的数据，并且可以拥有多个slave节点。这种设计不仅提升了数据的可用性和一致性，还使得集群能够处理大量的数据和请求。

分片集群采用哈希槽（hash slots）机制来实现数据的分区。具体来说，Redis分片集群共有16384个哈希槽，每个数据键通过CRC16校验后对16384取模来决定放置到哪个槽。每个节点负责一定数量的槽，并处理映射到这些槽上的键值数据。这种机制确保了数据能够均匀地分布到不同的节点上，从而实现负载均衡和高效访问。

Redis cluster集群节点最小配置6个节点以上（3主3从），其中主节点提供读写操作，从节点作为备用节点，不提供请求，只作为故障转移使用。

Redis cluster采用**虚拟槽分区**，所有的键根据哈希函数映射到0 ~ 16383个整数槽内，每个节点负责维护一部分槽以及槽所映射的键值数据。

工作原理：

1. 通过哈希的方式，将数据分片，每个节点均分存储一定哈希槽(哈希值)区间的数据，默认分配了16384个槽位
2. 每份数据分片会存储在多个互为主从的多节点上
3. 数据写入先写主节点，再同步到从节点(支持配置为阻塞同步)
4. 同一分片多个节点间的数据不保持一致性
5. 读取数据时，当客户端操作的key没有分配在该节点上时，redis会返回转向指令，指向正确的节点
6. 扩容时需要把旧节点的数据迁移一部分到新节点

Redis分片集群中数据是怎么存储和读取的？

Redis 集群引入了哈希槽的概念，有 16384 个哈希槽，集群中每个主节点绑定了一定范围的哈希槽范围，key通过对 16384 取模来决定放置哪个槽，通过槽找到对应的节点进行存储。

取值的逻辑是一样的。

[11] redis发布订阅

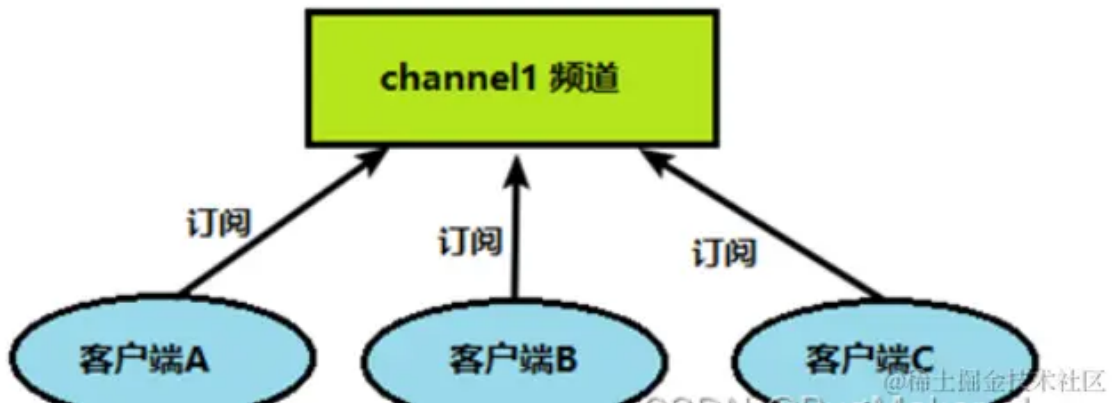
Redis 发布订阅 (pub/sub) 是一种消息通信模式：发送者 (pub) 发送消息，订阅者 (sub) 接收消息。

Redis 客户端可以订阅任意数量的频道。

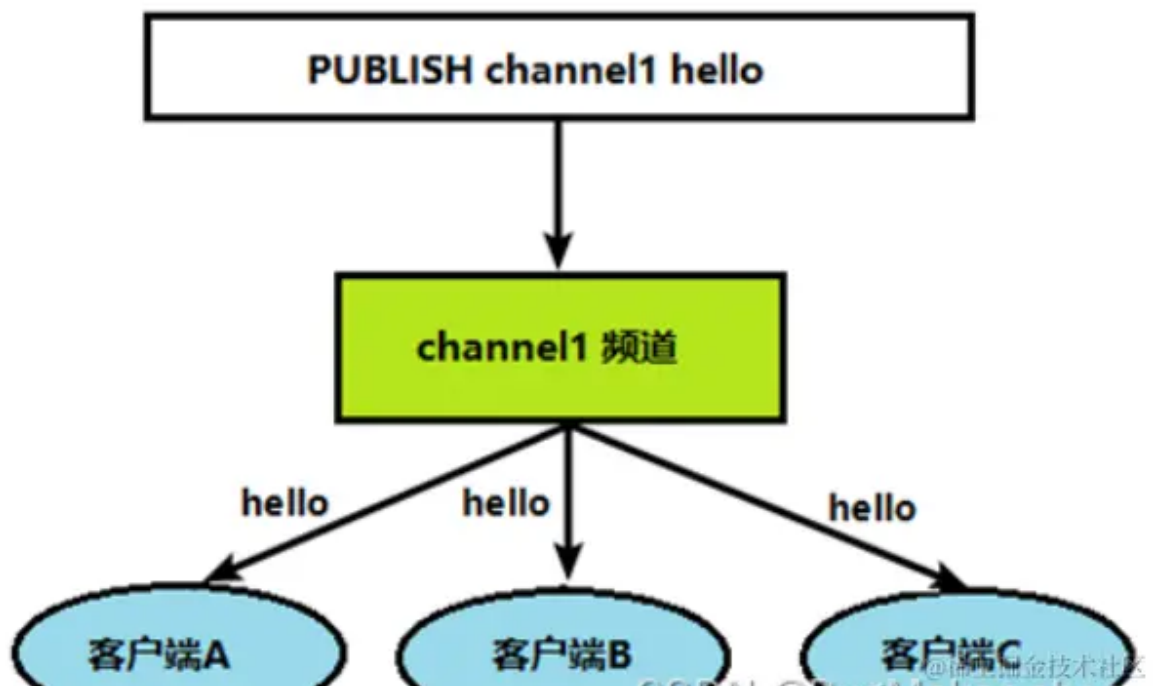
Redis 的 subscribe 命令可以让客户端订阅任意数量的频道，每当有新信息发送到被订阅的频道时，信息就会被发送给所有订阅指定频道的客户端。

Redis的发布和订阅

- (1) 客户端可以订阅频道如下图



(2) 当给这个频道发布消息后，消息就会发送给订阅这个频道的客户端



发布订阅命令行实践

(1) 打开一个客户端订阅channel1

```
SUBSCRIBE channel1
```

```
127.0.0.1:6379> subscribe channel1
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel1"
3) (integer) 1
```

(2) 打开另一个客户端，给channel1发布消息hello

```
publish channel1 hello
```

```
127.0.0.1:6379> publish channel1 hello
```

注：返回的1是订阅者数量

(3) 打开第一个客户端可以看到发送的消息

```
127.0.0.1:6379> subscribe channel1
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel1"
3) (integer) 1
1) "message"
2) "channel1"
3) "hello"
@稀土掘金技术社区
```

注：发布的消息没有持久化，如果在订阅的客户端收不到hello，只能收到订阅后发布的消息